COMPLETE DISCORD BOT TUTORIAL

Building a Bot from Scratch with Python

TABLE OF CONTENTS

1. INTRODUCTION AND PREREQUISITES

This tutorial will guide you through creating a fully functional Discord bot using Python and the discord.py library. We will build a bot with multiple features including general commands, moderation tools, and channel management.

What You Will Learn

- How to set up a Discord application

- Understanding the discord.py library structure

- Creating and organizing bot commands using Cogs

- Implementing moderation features

- Managing bot permissions and security

Prerequisites

- Python 3.8 or later

- A Discord account

- A Discord server for testing

- Basic Python knowledge

- VS Code or any text editor

2. DISCORD DEVELOPER SETUP

Before writing any code, you need to create a Discord application and get a bot token.

Step 1: Creating Your Application

1. Visit https://discord.com/developers/applications
2. Click 'New Application' button
3. Enter your bot name and click 'Create'
4. You now have an application ID in the General section

Step 2: Creating the Bot User

1. Go to the 'Bot' tab
2. Click 'Add Bot'
3. Your bot user is now created
4. Keep the default settings for now

Step 3: Getting Your Token

1. In the 'Bot' tab, click 'Reset Token' or view existing token
2. Copy the token (it will never be shown again!)
3. Store it securely - never commit it to GitHub
4. Always use a .env file to store sensitive data

3. PROJECT CONFIGURATION

Setting Up Your Project Structure

Create the following folder structure:

my_discord_bot/

main.py

general.py

moderation.py

channels.py

help_cog.py

.env

requirements.txt

Installing Dependencies

Create a requirements.txt file with:

discord.py==2.3.2

python-dotenv==1.0.0

Then install: pip install -r requirements.txt

Configuring Your Token

Create a .env file in your project root:

DISCORD_TOKEN=your_token_here

4. UNDERSTANDING COGS AND COMMANDS

What Are Cogs?

Cogs are a way to organize your bot commands into separate modules. Each cog is a class that inherits from commands.Cog. This keeps your code clean and makes it easy to manage large numbers of commands.

Basic Cog Structure

```
class MyCog(commands.Cog):
def __init__(self, bot):
self.bot = bot

@commands.command()
async def mycommand(self, ctx):
await ctx.send('Hello!')

async def setup(bot):
await bot.add_cog(MyCog(bot))
```

5. GENERAL COMMANDS EXPLAINED

Command: !ping

Purpose: Tests if the bot is responsive

How it works: When a user types !ping, the bot immediately responds with 'Pong!'

Code implementation:

```
@commands.command()
async def ping(self, ctx):
await ctx.send('Pong!')
```

Command: !hello

Purpose: Greets the user who called the command

How it works: The bot uses ctx.author.mention to get the user's mention and greets them

Code implementation:

```
@commands.command()
async def hello(self, ctx):
await ctx.send(f'Hello {ctx.author.mention}!')
```

Command: !info

Purpose: Displays server information

How it works: Uses Discord embeds to display formatted information about the server

Information shown:

- Server name and member count
- Server creation date
- Server owner

6. MODERATION COMMANDS EXPLAINED

Command: !kick @member [reason]

Purpose: Removes a member from the server temporarily

Permissions Required: Kick Members

How it works:

1. Checks if user has permission

2. Prevents kicking yourself or higher roles

3. Removes the member with optional reason

4. Sends confirmation embed to chat

Command: !ban @member [reason]

Purpose: Permanently bans a member from the server

Permissions Required: Ban Members

Key difference from kick: The member cannot rejoin unless unbanned

Command: !unban name#tag

Purpose: Reverses a ban and allows the member to rejoin

How it works:

1. Fetches the server's ban list

2. Searches for the member by name and discriminator

3. Removes the ban if found

Command: !mute @member [reason]

Purpose: Prevents a member from sending messages

How it works:

1. Looks for a role named 'Muted'

2. Adds this role to the member

3. The Muted role should have Send Messages permission disabled

Command: !unmute @member [reason]

Purpose: Removes the mute from a member

How it works: Removes the Muted role from the member

7. CHANNEL MANAGEMENT COMMANDS EXPLAINED

Command: !lock [#channel]

Purpose: Prevents members from sending messages in a channel

Permissions Required: Manage Channels

How it works:

1. Creates a permission overwrite for @everyone

2. Sets send_messages permission to False

3. Moderators can still send messages

4. Useful during moderation events or announcements

Command: !unlock [#channel]

Purpose: Re-enables message sending in a locked channel

How it works:

1. Removes the permission overwrite

2. Restores default permissions

3. Members can send messages again

Command: !lockall

Purpose: Locks ALL text channels on the server at once

Use case: Emergency lockdown during raids or spam attacks

How it works:

1. Loops through all text channels

2. Applies send_messages=False to each

3. Counts and reports number of channels locked

Command: !unlockall

Purpose: Unlocks ALL text channels at once

Use case: Restoring the server after emergency lockdown

8. ADVANCED TOPICS AND BEST PRACTICES

Permission Checks

Always verify user permissions before executing sensitive commands:

```
@commands.has_permissions(kick_members=True)
```

This prevents unauthorized users from using moderation commands

Error Handling

Always wrap Discord API calls in try-except blocks:

```
try:
await member.kick(reason=reason)
except discord.Forbidden:
await ctx.send('No permission')
```

Using Embeds for Better Output

Embeds provide formatted, colored messages that look professional:

```
embed = discord.Embed(title='Title', color=discord.Color.blue())
```

This is much better than plain text responses

Security Best Practices

- Never hardcode your token in source code
- Always use environment variables (.env files)
- Add .env and .gitignore to version control
- Regenerate your token if it's ever exposed
- Limit bot permissions to what's actually needed

CONCLUSION AND NEXT STEPS

You now have a fully functional Discord bot with multiple features. The modular structure using Cogs makes it easy to add more commands in the future.

Ideas for Expanding Your Bot

- Create a welcome message for new members
- Add a logging system for moderation actions
- Create fun commands (joke, quote, etc.)
- Implement database integration
- Add automated moderation (spam detection)
- Create role assignment commands

Resources

- discord.py Documentation: https://discordpy.readthedocs.io/
- Discord Developer Portal: https://discord.com/developers/
- Python Documentation: https://docs.python.org/3/